# Ders 19 Çalışma Özeti

## Ders 19 Çalışma Özeti

### Genel Konular

- Bu kayıt uygulama oturumudur.
  - Oturumu bir asistan yürütür. Aynı günün teorik dersinde (ders 18) işlenen konuların uygulamasıdır.
  - Kaydın ilk 00:06:40'ı bekleme kısmıdır.
- İş parçacıkları (thread)
  - Program, süreç (process) ve thread kavramları.
  - `extends Thread` ve `implements Runnable` ile thread oluşturma.
  - `start`, `join`, `setPriority`, `Thread.sleep`.
  - Yarış durumu (race condition) ve `synchronized`.
  - Thread ile paralel toplama ve süre ölçümü.
- Dosya işlemleri
  - `FileWriter` ile metin dosyasına yazma.
  - `Scanner` ile satır satır okuma, `split`, `Double.parseDouble`.
  - `ObjectOutputStream` / `ObjectInputStream` ile nesne yazma ve okuma. `NotSerializableException`.
- Tip dönüşümü (typecasting)
  - İlkel tipler arasında dönüşüm. Sınıflar arasında dönüşüm.
- İç sınıflar (inner class)

### Hocanın Özellikle Vurguladığı Kısımlar

- Thread'leri çalıştırmak için `run()` değil `start()` çağrılır. `run()` doğrudan çağrılırsa kod normal bir metot gibi, aynı akışta çalışır.
- Thread'lerin çalışma sırası belirsizdir (deterministik değildir). İşletim sisteminin ayırdığı kaynağa göre her çalıştırmada değişir.
- Thread oluşturmanın tercih edilen yolu `implements Runnable`'dır. Java'da tek `extends` hakkı vardır. Bu hakkı `Thread` için harcamak tasarımı kısıtlar.
- Aynı değişkene birden fazla thread erişiyorsa sonuç bozulabilir. Bankacılık, muhasebe gibi uygulamalarda bu kabul edilemez. Java'da çözüm `synchronized` anahtar kelimesidir.
- Thread ve süreç konuları İşletim Sistemleri dersinde (deadlock, mutual exclusion, starvation) ayrıntılı işlenir.
- Paralelleştirme işi hızlandırabilir ama hız thread sayısıyla doğru orantılı artmaz.
- Nesneyi dosyaya yazmak için sınıf `java.io.Serializable` arayüzünü uygulamalıdır.
- `double` → `int` dönüşümü yuvarlama yapmaz, ondalık kısmı atar.
- Konu önceliği: Bugün görülen bütün konular, thread dahil, önemlidir.

### Kısa Tekrar Notları

- Program: diskteki çalıştırılabilir dosya. Süreç: programın çalışan hâli. Her sürecin en az bir thread'i vardır.
- Şimdiye kadar yazılan programlar tek thread'liydi.
- `t.start()`: thread'i başlatır. Thread içinde `run()` çalışır.
- `t.join()`: ana akış, `t` bitene kadar bekler.
- `t.setPriority(n)`: öncelik verir. Garanti değildir.
- `Thread.sleep(4000)`: thread'i 4 saniye uyutur.

- `x++` atomik değildir. Oku-değiştir-yaz adımları arasında başka thread araya girebilir.
- `synchronized` metot: Aynı nesne üzerinde aynı anda yalnızca bir thread çalışabilir.
- `new Random(120)`: Aynı tohum (seed) her çalıştırmada aynı sayı dizisini üretir.
- Metin dosyası: `FileWriter.write`, `close`. Okuma: `new Scanner(new File(...))`, `hasNextLine`, `nextLine`.
- Nesne dosyası: `writeObject`, `readObject`. Hatalar: `IOException`, `ClassNotFoundException`.
- `A a = new B();` geçerlidir (B, A'dan türer). `B b = (B) new A();` derlenir ama çalışırken hata verir.
- İç sınıf, dış sınıf olmadan anlamı olmayan yapılar için kullanılır.

## Detaylı Açıklamalar

### Thread oluşturma

```
class SimpleThread extends Thread {
    private String name;
    SimpleThread(String name) { this.name = name; }
    public void run() {
        System.out.println(name + " başladı");
        for (int i = 0; i < 5; i++) System.out.println(name + " " + i);
        System.out.println(name + " bitti");
    }
}

SimpleThread t1 = new SimpleThread("1");
SimpleThread t2 = new SimpleThread("2");
t1.start(); t2.start();
t1.join(); t2.join();
System.out.println("Normalde son satır olması lazım");
```

- `join` yazılmadan çalıştırıldığında "son satır" yazısı en başta çıktı. `start` sonrası ana akış beklemeden devam eder.
- `Runnable` ile:

```
class SimpleRunnable implements Runnable {
    public void run() { /* ... */ }
}
Thread t1 = new Thread(new SimpleRunnable());
t1.start();
```

- Asistan, `Runnable`'ı önceki uygulamadaki `Comparable` arayüzüne benzetti. Arayüzü uygulayan sınıf `run` metodunu yazmak zorundadır.

### Yarış durumu: statik sayaç

- Statik bir sayı 5'ten başlar. İki thread sayıyı artıran metodu çağırır. Metot sayıyı bir artırıp yazdırır.
- Çıktı bazen "6, 7", bazen "7, 7" oldu.
- Sebep: Birinci thread sayıyı 6 yapar ama yazdırmadan önce ikinci thread sayıyı 7 yapar. İki thread de 7 yazdırır.

### Kitapçı simülasyonu ve synchronized

- `Kitapci` sınıfında bakiye ve kitap sayısı var. `kitapAl` kitap sayısını artırır, bakiyeyi azaltır. `kitapSat` tersini yapar.
- Başlangıç: bakiye 100 TL, 100 kitap. Alış ve satış fiyatı 10 TL. Bir thread 10 kitap alır, diğer thread 10 kitap satar.
- Beklenen sonuç her zaman bakiye 100, kitap 100'dür.
- Gecikme 10 ms yapılınca sonuçlar bozuldu. Bir denemede son bakiye 120, kitap sayısı 100 çıktı (00:21:07).

- İki metot `synchronized` yapılınca sonuç her seferinde 100 / 100 çıktı.
- Derstekine benzer kısa bir deneme (Java ile çalıştırıldı):

```
class Kitapci {
    double bakiye = 100; int kitap = 100;
    synchronized void al(double f) { kitap++; bakiye -= f; }
    synchronized void sat(double f) { kitap--; bakiye += f; }
}
// synchronized olmadan bir çalıştırmada: bakiye=20.0 kitap=101
// synchronized ile: bakiye=100.0 kitap=100
```

### Paralel toplama

- Rastgele tam sayılardan oluşan büyük bir `ArrayList` oluşturulur. Aynı sayıların toplanması için `Random`'a sabit tohum verilir.
- Tek thread ile toplama yaklaşık 150 ms sürdü.
- Liste beş parçaya bölündü. Her parçayı ayrı bir thread topladı. Sonra beş kısmi toplam birleştirildi. Süre yaklaşık 86-92 ms oldu.
- Beş thread kullanıldığı hâlde süre beşte bire düşmedi. Sebepler:
  - Thread oluşturmanın kendi maliyeti vardır.
  - `join` en yavaş thread'i bekler.
- Asistanın örneği: Oyunlarda karakterlerin aynı anda hareket etmesi paralel çalışma gerektirir.

### Metin dosyasına yazma ve okuma

```
FileWriter myWriter = new FileWriter("urunler.txt");
for (Urun u : urunler)
    myWriter.write(u.getIsim() + ", " + u.getFiyat() + "\n");
myWriter.close();

Scanner reader = new Scanner(new File("urunler.txt"));
while (reader.hasNextLine()) {
    String[] tokens = reader.nextLine().split(", ");
    urunler.add(new Urun(tokens[0], Double.parseDouble(tokens[1])));
}
reader.close();
```

- Satırdan okunan her şey `String`'dir. Fiyatla işlem yapmak için `Double.parseDouble` ile dönüştürülür.
- Dosya adı yanlış verilince `catch` bloğuna düşüldü ve "dosya bulunamadı" yazıldı. Program `catch` sonrası devam etti, liste boş yazdırıldı.

### Nesneyi dosyaya yazma

- `Urun` sınıfı `Serializable` olmadığı için `writeObject` çağrısı `java.io.NotSerializableException` verdi.
- Aynı sınıf `implements Serializable` ile yazılınca nesne başarıyla yazıldı. Dosya uzantısı (`.ser`, `.txt` veya hiç) fark etmez.
- Dosya metin editöründe açılınca okunamayan karakterler görünür. Formatı Java belirler.
- Okurken sınıf bulunamazsa `ClassNotFoundException` oluşur.
- Öğrenci sorusu: Birden fazla nesne nasıl yazılır? Asistanın cevabı: Nesneleri bir veri yapısında (örneğin `LinkedList`) toplayıp tek seferde yazın. Okurken de tek seferde okunur.

### Tip dönüşümü

- `int` → `double` otomatiktir: `911` → `911.0`.
- `double` → `int` için açık dönüşüm gerekir: `(int) 5.9` → `5`. Yuvarlama yapılmaz.
- `char` ile `int` arasında ASCII değerine göre dönüşüm yapılır.
- Sınıflarda: `B extends A` ve `B`, `printFromA`'yı override ediyor.
  - `A a1 = new B();` geçerli. `a1.printFromA()` B'deki sürümü çalıştırır.

- `a1.printFromB()` derlenmez. Referans tipi `A`'dır.
- `((B) a1).printFromB()` çalışır.
- `B b = (B) new A();` derlenir ama çalışırken `ClassCastException` verir.

### İç sınıflar

- Bir sınıfın içinde başka bir sınıf tanımlanabilir. İç sınıf `private` veya `static` olabilir.
- Kalıtım ile karıştırılmamalı. Kalıtımda alt sınıf üst sınıfın bütün özelliklerini taşır ve ekler. İç sınıfta iç nesne, dış nesne olmadan anlam taşımaz.
- Örnek: Önceki uygulamadaki kitaplık-raf örneğinde "kitaplık olmadan raf olmaz" deniyorsa `Raf`, `Kitaplik` içinde iç sınıf olarak yazılabilir. Böylece `main`'de tek başına `Raf` oluşturulamaz.

### Ders sonrası notlar

- `setPriority` değeri 1 ile 10 arasındadır (`Thread.MIN_PRIORITY` ile `Thread.MAX_PRIORITY`). İşletim sistemi bu değeri dikkate almak zorunda değildir.
- Statik sayaç örneğinde 6-6 çıktısı da mümkündür: İki arttırmadan biri kaybolur. Java ile 20000 denemede 10 kez görüldü.
- `synchronized` metot, kilidi metodun ait olduğu nesne üzerinden alır. Farklı `Kitapci` nesneleri birbirini beklemez. Statik değişkeni koruyan `synchronized` metot `static` olmalıdır, yoksa her nesne kendi kilidini kullanır.
- Aynı akışa birden çok kez `writeObject` çağrılabilir; okurken aynı sırayla `readObject` yapılır (Java ile denendi). Hocanın "tek seferde" önerisi tek yol değildir.
- Nesne dosyasının formatını işletim sistemi değil Java belirler. Yazılan dosyanın ilk baytları `ac ed 00 05` çıktı (Java ile denendi).
- Paralel toplamada hız kazancı makinenin çekirdek sayısıyla da sınırlıdır.
- Süre ölçümünde listenin oluşturulması ölçüme dahil edilmemelidir. Yalnızca toplama kısmı `System.currentTimeMillis()` (veya `System.nanoTime()`) çağrıları arasına alınmalıdır.
- `Scanner(File)` yapıcısı `FileNotFoundException` fırlatır. Bu yüzden `try-catch` içinde ya da `throws` ile kullanılmalıdır.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.